JobMatch AI — Build Specification

> **Purpose:** This document is the complete build spec for an AI-powered job search assistant. Read it end-to-end before writing any code. Build it in the order given. Do not skip phases.

---

1. Product Overview

What it is: A web app where a user uploads their CV, enters search criteria, and gets a table of matching job postings from LinkedIn, Indeed, Glassdoor, and free job APIs — ranked by CV fit — with direct apply links.

What v1 is NOT:

- No auto-apply
- No CV rewriting/tailoring (that's v2)
- No saved searches, email digests, or background refreshes (v2)
- No team/multi-user features

Key constraint — BYOK (Bring Your Own Keys): Users provide their own Apify token and Anthropic API key. The app never pays for third-party API usage. Keys are encrypted at rest.

---

2. Tech Stack (locked — do not substitute)

Layer	Choice	Why
Framework	**Next.js 15 (App Router)** + TypeScript	Full-stack in one repo, Vercel-native
UI	**Tailwind CSS v4** + **shadcn/ui**	Fast, clean, production-quality
Auth	**Supabase Auth** (email magic link)	Integrates with Postgres RLS
Database	**Supabase Postgres**	Managed, free tier generous
ORM	**Drizzle ORM**	TypeScript-first, lighter than Prisma, works great on Vercel Edge/Node
File storage	**Supabase Storage**	For CV uploads (PDF)
LLM	**Anthropic SDK** (`@anthropic-ai/sdk`), model `claude-sonnet-4-5`	For CV parsing + job ranking

Layer	Choice	Why
Scraping	**Apify Client** (`apify-client`)	LinkedIn, Indeed, Glassdoor
Free job APIs	**Adzuna**, **JSearch** (RapidAPI), **Remotive**	Supplement Apify, broader coverage
CV parsing	**pdf-parse** for text extraction, then Claude for structuring	
Validation	**Zod**	Runtime validation everywhere
Forms	**react-hook-form** + Zod resolver	
Encryption	**Node crypto** (AES-256-GCM) for user API keys	
Deployment	**Vercel** (app) + **Supabase** (db/auth/storage)	

Do NOT add: Prisma, NextAuth, tRPC, Redux, any ORM other than Drizzle, any UI library other than shadcn/ui + Tailwind, Playwright, or any scraping library. Apify handles scraping.

---

3. Architecture

```
┌──────────────────────────────────────────────────────────┐
│  Browser (Next.js client)                                │
│  - Upload CV, enter search, view results                 │
└──────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
│  Next.js Server (App Router route handlers + Server Actions) │
│  - /api/cv/upload       → parse CV with Claude           │
│  - /api/search          → orchestrate job search         │
│  - /api/keys            → encrypt + store user API keys  │
└──────────────────────────────────────────────────────────┘
          │                 │                 │
          ▼                 ▼                 ▼
   ┌──────────┐      ┌──────────┐      ┌──────────┐
   │ Supabase │      │ Anthropic │      │ Apify + free │
   │ Postgres │      │ API (Claude) │   │ job APIs │
   │ Storage  │      │ (user's key) │   │ (user's key) │
```

```
      |  Auth        |    └──────────────┘    └──────────────────┘
      └──────────────┘
```

Search flow (the core loop):

```
User clicks "Search Jobs"
  ↓
Server reads user's stored CV structured JSON
  ↓
Server calls free APIs in parallel (Adzuna, JSearch, Remotive)
  ↓
Server triggers Apify Actors in parallel (LinkedIn, Indeed, Glassdoor)
  - Uses `.call()` (synchronous run + wait for results)
  - Each with a timeout of 90s
  - If one fails, continue with the rest
  ↓
Deduplicate results by (normalized title + normalized company)
  ↓
Send jobs + CV summary to Claude for ranking
  - Claude returns match_score (0-100) + one-line reason per job
  - Batched: 20 jobs per Claude call
  ↓
Persist search + results to DB (so user can revisit without re-scraping)
  ↓
Return results to client, sorted by match_score DESC
```

Why no background jobs / queue? Vercel serverless functions have a 60s default / 300s max (Pro) timeout. Apify `.call()` with a 90s timeout fits. We run the whole search in a single request, show a loading state, and cache aggressively. No Redis, no BullMQ, no cron — keeps infra simple for v1.

---

4. Database Schema (Drizzle)

Create in `src/db/schema.ts`.

```typescript
```

```
// users — Supabase Auth manages this table; we reference auth.users.id

// profiles — our app-level user data
profiles
  id              uuid PK (= auth.users.id)
  email           text
  created_at      timestamptz default now()

// user_api_keys — encrypted BYOK storage
user_api_keys
  user_id         uuid PK FK → profiles.id
  anthropic_key   text (encrypted, nullable)
  apify_token     text (encrypted, nullable)
  adzuna_app_id   text (nullable, not secret enough to encrypt)
  adzuna_app_key  text (encrypted, nullable)
  rapidapi_key    text (encrypted, nullable)  // for JSearch
  updated_at      timestamptz default now()

// cvs — one active CV per user (v1); schema allows multiple for v2
cvs
  id              uuid PK default gen_random_uuid()
  user_id         uuid FK → profiles.id
  file_path       text           // Supabase Storage path
  raw_text        text           // extracted PDF text
  structured      jsonb          // Claude-parsed: name, skills[], experience[], edu
  is_active       boolean default true
  created_at      timestamptz default now()

// searches — a single search run
searches
  id              uuid PK default gen_random_uuid()
  user_id         uuid FK → profiles.id
  cv_id           uuid FK → cvs.id
  query           text           // job title / keywords
  location        text
  remote_only     boolean default false
  sources         text[]         // ["linkedin", "indeed", "adzuna", ...]
  status          text           // "running" | "complete" | "failed"
  error           text nullable
  created_at      timestamptz default now()
  completed_at    timestamptz nullable

// job_results — individual jobs per search
```

```
job_results
  id              uuid PK default gen_random_uuid()
  search_id       uuid FK → searches.id ON DELETE CASCADE
  source          text           // "linkedin" | "indeed" | "glassdoor" | "adzuna" |
  source_job_id   text nullable  // if provided by source
  title           text
  company         text
  location        text nullable
  remote          boolean default false
  salary          text nullable
  posted_at       timestamptz nullable
  description     text nullable
  apply_url       text NOT NULL
  match_score     integer nullable  // 0-100, set after Claude ranks
  match_reason    text nullable
  created_at      timestamptz default now()

  INDEX on (search_id, match_score DESC)
  UNIQUE (search_id, source, source_job_id) -- dedupe
```

Row Level Security (RLS): Enable on all tables. Policy: `user_id = auth.uid()`. Write Drizzle migrations for tables; write raw SQL in `supabase/policies.sql` for RLS policies.

---

5. Project Structure

```
jobmatch-ai/
├── src/
│   ├── app/
│   │   ├── (auth)/
│   │   │   ├── login/page.tsx
│   │   │   └── callback/route.ts
│   │   ├── (app)/
│   │   │   ├── layout.tsx              # protected, requires auth
│   │   │   ├── dashboard/page.tsx      # home after login
│   │   │   ├── cv/page.tsx             # upload + view CV
│   │   │   ├── search/page.tsx         # search form + results
│   │   │   ├── search/[id]/page.tsx    # view past search
│   │   │   └── settings/page.tsx       # manage API keys
│   │   ├── api/
│   │   │   ├── cv/
│   │   │   │   ├── upload/route.ts
```

```
│   │   │   │   └── parse/route.ts
│   │   │   ├── search/
│   │   │   │   ├── route.ts            # POST: run a new search
│   │   │   │   └── [id]/route.ts       # GET: fetch search + results
│   │   │   └── keys/route.ts           # POST/GET user API keys
│   │   ├── layout.tsx
│   │   └── page.tsx                    # landing page
│   ├── components/
│   │   ├── ui/                         # shadcn components
│   │   ├── cv-uploader.tsx
│   │   ├── search-form.tsx
│   │   ├── results-table.tsx
│   │   └── api-keys-form.tsx
│   ├── lib/
│   │   ├── supabase/
│   │   │   ├── client.ts               # browser client
│   │   │   ├── server.ts               # server client
│   │   │   └── middleware.ts
│   │   ├── anthropic.ts                # wrapper: takes user's key
│   │   ├── apify.ts                    # wrapper: takes user's token
│   │   ├── crypto.ts                   # encrypt/decrypt API keys
│   │   ├── cv-parser.ts                # PDF → text → Claude → structured
│   │   ├── job-sources/
│   │   │   ├── index.ts                # unified runJobSearch()
│   │   │   ├── adzuna.ts
│   │   │   ├── jsearch.ts
│   │   │   ├── remotive.ts
│   │   │   ├── apify-linkedin.ts
│   │   │   ├── apify-indeed.ts
│   │   │   └── apify-glassdoor.ts
│   │   ├── matcher.ts                  # Claude-based job ranking
│   │   ├── dedupe.ts
│   │   └── types.ts                    # shared TS types
│   ├── db/
│   │   ├── schema.ts                   # Drizzle schema
│   │   └── index.ts                    # db client
│   └── middleware.ts                   # Supabase session refresh
├── supabase/
│   ├── migrations/                     # drizzle-kit output
│   └── policies.sql                    # RLS policies
├── drizzle.config.ts
├── next.config.ts
├── tsconfig.json
```

```
├── package.json
└── .env.local.example
```

---

6. Environment Variables

Create `.env.local.example` with these keys (real values go in `.env.local`):

```bash
# Supabase
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=                # server-only, for admin ops
DATABASE_URL=                             # direct Postgres URL for Drizzle

# App encryption — used to encrypt user API keys at rest
# Generate with: openssl rand -base64 32
APP_ENCRYPTION_KEY=

# App
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

No third-party API keys in env — users provide their own.

---

7. External API Integration Details

7.1 Anthropic (user's key)

- Model: `claude-sonnet-4-5`
- Use `@anthropic-ai/sdk`
- Always instantiate per-request with user's decrypted key: `new Anthropic({ apiKey: userKey })`
- Never log keys. Never return keys to client.

7.2 Apify Actors

Use `apify-client`. Instantiate per-request with user's token.

Recommended Actors (document the exact IDs in code comments; they are stable community Actors):

- LinkedIn: `bebity/linkedin-jobs-scraper` or `curious_coder/linkedin-jobs-scraper`
- Indeed: `misceres/indeed-scraper`
- Glassdoor: `bebity/glassdoor-jobs-scraper`

Build it so the Actor ID is configurable per source (env-driven with sane defaults), because Actor IDs change and deprecate.

Call pattern:

```typescript
const run = await apifyClient.actor(actorId).call(inputPayload, {
  timeout: 90,        // seconds
  memory: 1024,
});
const { items } = await apifyClient.dataset(run.defaultDatasetId).listItems();
```

Wrap each source call in `Promise.allSettled` so one failure doesn't kill the whole search.

7.3 Free APIs

- **Adzuna:** `https://api.adzuna.com/v1/api/jobs/{country}/search/1` — needs `app_id` + `app_key` (free signup)
- **JSearch** (RapidAPI): `https://jsearch.p.rapidapi.com/search` — needs RapidAPI key (free tier: 200/month)
- **Remotive:** `https://remotive.com/api/remote-jobs` — no key needed, remote-only jobs

Normalize all sources to this shape in `lib/types.ts`:

```typescript
```

```typescript
export type NormalizedJob = {
  source: JobSource;
  sourceJobId?: string;
  title: string;
  company: string;
  location?: string;
  remote: boolean;
  salary?: string;
  postedAt?: Date;
  description?: string;
  applyUrl: string;   // required
};
```

---

8. Core Algorithms

8.1 CV Parsing (`lib/cv-parser.ts`)

```
1. Receive PDF file
2. Extract raw text with pdf-parse
3. Send to Claude with this system prompt:
   "Extract structured data from this CV. Return ONLY valid JSON matching this
schema:
   { name, email, phone, location, summary, skills: string[],
     experience: [{title, company, startDate, endDate, description}],
     education: [{degree, institution, year}],
     languages: string[] }"
4. Parse JSON, validate with Zod
5. Store raw_text + structured in cvs table
```

8.2 Deduplication (`lib/dedupe.ts`)

```
Key = normalize(title) + "|" + normalize(company) + "|" + normalize(location ??
"")
where normalize = lowercase, strip punctuation, collapse whitespace.
Keep first occurrence; preserve source priority: linkedin > indeed > glassdoor >
others
(i.e., if we see LinkedIn first, keep it; if we see Remotive first then LinkedIn,
prefer LinkedIn).
```

8.3 Job Matching (`lib/matcher.ts`)

```
Input: NormalizedJob[] + structured CV
Batch jobs in groups of 20
For each batch, call Claude:
  System: "You are a career advisor. Score how well each job matches the
candidate's CV
           from 0-100, and give a one-sentence reason. Return JSON array
           [{id, score, reason}]. Be honest; reject poor matches with low scores."
  User: { cv: {...}, jobs: [{id, title, company, description}, ...] }
Merge scores back onto jobs, sort DESC by score, persist to job_results.
```

Use Claude's tool use / structured output, or enforce JSON via prompt + Zod validation + retry on parse failure (max 2 retries).

8.4 Encryption (`lib/crypto.ts`)

- Algorithm: AES-256-GCM
- Key: `APP_ENCRYPTION_KEY` (base64, 32 bytes)
- Format stored in DB: `<iv_hex>:<authTag_hex>:<ciphertext_hex>`
- Export `encrypt(plaintext): string` and `decrypt(ciphertext): string`
- Never log plaintext or ciphertext

---

9. UI Pages

Keep design clean and minimal. Use shadcn/ui components exclusively. Tailwind for layout.

Landing (`/`)

Headline, one-paragraph pitch, "Get Started" → login.

Login (`/login`)

Email input → Supabase magic link.

Dashboard (`/dashboard`)

Three cards:

1. **CV** — "Upload CV" or "View current CV"
2. **API Keys** — status (✓ Anthropic set, ✗ Apify missing, etc.)
3. **New Search** — CTA, disabled if CV or required keys missing.

CV (`/cv`)

Dropzone for PDF upload → shows parsing progress → displays structured CV with edit capability (v2). v1: just display parsed JSON in a readable card layout.

Settings (`/settings`)

Form with masked inputs for each API key. Show "••••••••" if set, "Not configured" if not. Save button per key. Include a "How to get this key" help link per provider.

Search (`/search`)

Form:

- Job title / keywords (required)
- Location (optional)
- Remote only (checkbox)
- Sources (multiselect, default: all available based on which keys user has set)
- "Search" button

On submit: show loading state with per-source progress (e.g., "Searching LinkedIn... ✓", "Searching Indeed... ⏳"). When complete, render `ResultsTable`.

Results Table (`components/results-table.tsx`)

Columns: Match Score | Title | Company | Location | Source | Posted | Apply

- Sort by match score DESC by default
- Sortable columns
- "Apply" is a link that opens `apply_url` in a new tab
- Expandable row to show match reason + description snippet
- Filter bar: source, min match score, remote-only

Search history (`/search/[id]`)

Same ResultsTable, read from DB, no re-scraping.

---

10. Build Phases

Build in this order. **Do not proceed to the next phase until the current one is verified working.**

Phase 1 — Foundation (day 1)

1. `npx create-next-app@latest jobmatch-ai --typescript --tailwind --app --eslint`
2. Install: `@supabase/supabase-js`, `@supabase/ssr`, `drizzle-orm`, `drizzle-kit`, `postgres`, `zod`, `react-hook-form`, `@hookform/resolvers`, `@anthropic-ai/sdk`, `apify-client`, `pdf-parse`
3. Install shadcn: `npx shadcn@latest init`, then add: `button`, `input`, `form`, `card`, `table`, `dialog`, `sonner`, `badge`, `tabs`
4. Set up Supabase project (dashboard), copy env vars to `.env.local`
5. Write Drizzle schema (Section 4)
6. `drizzle-kit push` to create tables
7. Write + apply RLS policies (`supabase/policies.sql`)
8. Wire Supabase Auth middleware + magic-link login
9. Protected route group `(app)` that redirects to `/login` if not authed
10. **Verify:** can sign up, log in, see empty dashboard.

Phase 2 — API Keys (day 1-2)

1. Build `lib/crypto.ts` with encrypt/decrypt + unit test (hardcoded input → known output after roundtrip)
2. Build `/api/keys` route (POST to save encrypted; GET to return which keys are set, NOT the values)
3. Build `/settings` page with form
4. **Verify:** save keys, check DB has ciphertext, round-trip decrypts correctly.

Phase 3 — CV Upload & Parse (day 2-3)

1. Supabase Storage bucket `cvs`, with RLS so users can only access their own files
2. `/api/cv/upload` — accept PDF, validate size (<5MB) + type, upload to storage, extract text with pdf-parse, call Claude with user's key, store structured JSON in DB
3. `/cv` page with uploader + parsed display
4. **Verify:** upload a real CV, see structured JSON populate correctly.

Phase 4 — Job Sources (day 3-5)

Build each source module in `lib/job-sources/`. Each exports `search(params, userKeys): Promise<NormalizedJob[]>`.

Order of implementation (easiest first, so you can test the pipeline end-to-end before Apify):

1. Remotive (no key needed)
2. Adzuna
3. JSearch
4. Apify-LinkedIn
5. Apify-Indeed
6. Apify-Glassdoor

Build `lib/job-sources/index.ts` with `runJobSearch(params, userKeys)` that:

- Calls all enabled sources in parallel with `Promise.allSettled`
- Returns `{ results: NormalizedJob[], errors: Record<Source, string> }`

Verify after each source: write a throwaway script `scripts/test-source.ts` that calls the source with test params and logs results.

Phase 5 — Matching + Search Endpoint (day 5-6)

1. `lib/matcher.ts` — Claude-based ranking (Section 8.3)
2. `/api/search` POST — the full pipeline:
    - Auth check
    - Load CV + decrypted keys
    - Create `searches` row with status=running
    - Call `runJobSearch`
    - Dedupe
    - Call matcher
    - Bulk insert into `job_results`
    - Update `searches` with status=complete
    - Return search id + results
3. `/api/search/[id]` GET — return search + results with RLS enforcement
4. **Verify:** hit endpoint with Postman/curl, see ranked results.

Phase 6 — Search UI (day 6-7)

1. `/search` page with form + loading state
2. `ResultsTable` component with sort/filter

3. `/search/[id]` history view
4. Dashboard "recent searches" list
5. **Verify:** full user flow works end-to-end in the browser.

Phase 7 — Polish & Deploy (day 7)

1. Error boundaries on every page
2. Toast notifications for all mutations (sonner)
3. Loading skeletons
4. Empty states for each page
5. "How to get your API keys" documentation page (`/help`) with step-by-step for each provider
6. Deploy to Vercel, point Supabase at production URL for redirects
7. **Verify in production:** full flow with a real CV and real keys.

---

11. Non-Negotiables (do not violate)

1. **RLS on every table.** Never return data bypassing user ownership.
2. **API keys are encrypted at rest.** Never in plaintext in DB, logs, or client.
3. **Never send user's API keys to the browser** after they're saved. The settings page shows masked placeholders and lets users overwrite, not read.
4. **All Anthropic/Apify calls happen server-side.** Keys never touch the client.
5. **Validate every input with Zod** at API boundaries.
6. **Handle partial failures.** If 2 of 6 sources fail, return results from the 4 that worked + surface which failed.
7. **No `any` types.** Strict TypeScript throughout.
8. **Idempotent searches.** Same search params within 10 minutes should return cached results — add a dedupe check by (user_id, query, location, sources hash) before running a new search.

---

12. Testing Checklist Before "Done"

- [ ] User can sign up with a new email
- [ ] User can set all three types of keys (Anthropic, Apify, Adzuna + JSearch)

- [ ] Keys are encrypted in the DB (verify in Supabase table editor)
- [ ] User uploads a 2-page PDF CV, structured JSON is correct
- [ ] Search with only free sources (Remotive) returns results
- [ ] Search with Apify-LinkedIn returns results
- [ ] Search with all sources returns deduplicated, ranked results
- [ ] If Apify token is invalid, search still completes with free sources + shows error for LinkedIn
- [ ] Match scores look reasonable (senior jobs score lower for junior CVs, etc.)
- [ ] Apply links open the correct external page
- [ ] Revisiting `/search/[id]` shows cached results, no re-scrape
- [ ] Another user cannot access my searches (test with two accounts)
- [ ] Production deploy on Vercel works

---

13. Known Limitations to Document in README

- LinkedIn/Indeed scraping via Apify may occasionally fail when source sites change; user's Apify quota is consumed.
- Claude match scores are subjective and should guide, not decide.
- Free API tiers are limited (JSearch 200/mo, Adzuna unmetered but throttled).
- Apify run timeout is 90s per source; very broad searches may be truncated.
- Scraping LinkedIn/Indeed may violate their ToS — tool is for personal use.

---

14. v2 Ideas (do NOT build in v1)

- AI CV tailoring per job (rewrite CV for specific JD, generate tailored PDF)
- Saved searches with scheduled background refresh (would need cron + queue)
- Email digest of new matches
- Cover letter generation
- Application tracker (Kanban: Saved / Applied / Interviewing / Offer / Rejected)
- Browser extension for one-click save from any job site
- Team/org accounts

End of spec. Build Phase 1 now. Do not deviate from the stack or schema without asking first.